AI guidelines for RoboDurden/Hoverboard-Firmware-Hack-Gen2.x-GD32

This document is written for AI coding agents and human reviewers who need to understand the firmware before changing it. The goal is not only to explain where things are, but also to protect the structure of a compact, timing-sensitive embedded firmware that supports many hoverboard board layouts and MCU targets.

The firmware code lives mainly in `HoverBoardGigaDevice/Inc/` and `HoverBoardGigaDevice/Src/`. The `Arduino Examples/` and `PlatformIO Examples/` folders are useful when working on UART control from ESP32, Arduino, ROS2, or similar external controllers.

Discussion and user reports still happen mainly in the original repository:

- `https://github.com/RoboDurden/Hoverboard-Firmware-Hack-Gen2.x/issues`
- `https://github.com/RoboDurden/Hoverboard-Firmware-Hack-Gen2.x/wiki`

The actual GD32 firmware code was moved into:

- `https://github.com/RoboDurden/Hoverboard-Firmware-Hack-Gen2.x-GD32`

1. First rules for AI agents

Read this section before editing code.

1. Do not flatten the architecture into one large procedural patch. The firmware intentionally uses an object-oriented-like structure in C.
2. Prefer adding a new `RemoteXY.c/.h`, `PilotXY.c/.h`, `bldcXY.c/.h`, layout define file, or target abstraction instead of modifying `main.c`, `bldc.c`, `it.c`, or `setup.c`.
3. Treat `main.c`, `bldc.c`, `it.c`, `setup.c`, `defines.h`, and `target.h` as core infrastructure. Change them only when the feature cannot fit into the existing extension points, when fixing a bug, or when improving the architecture for all modules.
4. Keep end-user configuration in `Inc/config.h` or a selected private `Inc/configDebug.h`. Many users cannot program; they must be able to select the firmware behavior with defines.

5. Keep the code small. Some supported boards have 32 kB flash MCUs. Avoid unnecessary tables, printf-heavy code, floating-point work in fast paths, dynamic allocation, broad abstractions, and linked-in alternatives that are not selected.
6. Respect the build matrix. Keil, PlatformIO, and the online compiler path must remain possible. Targets and layouts are selected through preprocessor symbols.
7. Never assume a pin, timer, DMA channel, interrupt name, or peripheral API is identical across targets. Put cross-target differences into `target.h` or a layout define file.
8. Never put slow parsing, logging, blocking waits, or heavy math in high-frequency interrupts unless the existing design already does it for a proven reason.
9. Be careful around safety. This firmware drives real motors and power electronics. Failsafe behavior, current limits, hall validity checks, charger detection, timeouts, and watchdog reloads are not decoration.
10. If you are not explicitly asked to refactor, do not refactor unrelated files. Small, local, boring changes are usually the best changes here.

2. Short history and naming

This firmware was originally forked from:

- `https://github.com/krisstakos/Hoverboard-Firmware-Hack-Gen2.1`

That original fork supported one specific board which is now classified as layout `2.1.2` in the Gen2.target.layout naming scheme. The old code also had Bluetooth support in `commsBluetooth.c`; this repository currently has Bluetooth references commented out in places such as `main.c` and `it.c`, but no active Bluetooth remote module. If Bluetooth is reintroduced later, it should fit the current remote architecture, for example as `RemoteBluetooth.c/.h`, instead of restoring old communication code directly into the core files.

The name `Gen2.x` originally referred to second-generation split hoverboard boards. Because the firmware now supports multiple targets and layouts, the mental model is closer to `Gen2.target.layout`, often written here as `Gen2.t.l`.

The first large architectural expansion was support for many board layouts. These are now represented by the many files in `HoverBoardGigaDevice/Inc/defines/`, for example `defines_2-1-20.h`.

The second large expansion was support for several MCU targets, currently centered on:

- `TARGET=1`: `GD32F130`
- `TARGET=2`: `GD32F103`, with optional `STM32F103` behavior
- `TARGET=3`: `GD32E230`

There is also partial work for `TARGET=4` / `MM32SPIN05` in the project files, `target.h`, and `defines_2-4-1.h`. Future wishes include targets such as MM32SPIN0x, LKS32, MM32SPIN25, and GD32E235. Add new targets by extending the target abstraction, not by scattering target-specific code through the firmware.

3. Build platforms

The firmware is intended to remain buildable through several paths:

- Keil uVision project files under `HoverBoardGigaDevice/`
- PlatformIO via `HoverBoardGigaDevice/platformio.ini`
- The experimental online compiler at `https://pionierland.de/hoverhack/`

Keil target selection sets preprocessor symbols such as `GD32F130` and `TARGET=1` in the target options. PlatformIO sets the same kind of symbols in `platformio.ini`, for example:

```
build_flags =
    -D GD32F103
    -D TARGET=2
```

The online compiler may not provide these predefined symbols. For that reason `Inc/defines.h` contains a fallback block that defines `TARGET` and the matching MCU symbol when `TARGET` is not already defined. Do not remove this fallback; the online compiler depends on it.

4. Main repository map

Important firmware folders and files:

- `HoverBoardGigaDevice/Inc/defines.h` : central include and configuration validation entry point.
- `HoverBoardGigaDevice/Inc/target.h` : target-specific API adaptation and Arduino-style pin helpers.
- `HoverBoardGigaDevice/Inc/configSelect.h` : selects normal `config.h` or private `configDebug.h`.
- `HoverBoardGigaDevice/Inc/config.h` : user-facing firmware configuration.
- `HoverBoardGigaDevice/Inc/defines/defines_2-t-l.h` : board layout pin maps and layout-specific options.
- `HoverBoardGigaDevice/Src/main.c` : startup, main loop, remote/pilot calls, master/slave mixing, battery and shutdown behavior.
- `HoverBoardGigaDevice/Src/setup.c` : clocks, watchdog, GPIO, PWM, ADC, DMA, USART, flash config storage.

- `HoverBoardGigaDevice/Src/it.c` : interrupts, SysTick, timeout timer, timer/ADC/DMA/USART interrupt glue.
- `HoverBoardGigaDevice/Inc/bldc.h` and `Src/bldc.c` : BLDC base interface and fast motor-control calculation.
- `HoverBoardGigaDevice/Src/driver.c` : PID and driving-mode interpretation.
- `HoverBoardGigaDevice/Inc/remote.h` and `Src/remote*.c` : selected remote input implementation.
- `HoverBoardGigaDevice/Inc/Pilot*.h` and `Src/Pilot*.c` : optional user/application logic.
- `HoverBoardGigaDevice/Src/comms*.c` : low-level serial sending, CRC, and master/slave communication.
- `Arduino Examples/` and `PlatformIO Examples/` : external-controller examples, especially UART and UART bus control.

5. Include and configuration chain

Almost every source file reaches the configuration through `Inc/defines.h` . The order matters.

The important flow is:

1. `defines.h` ensures `TARGET` and the MCU symbol exist if the build system did not provide them.
2. `defines.h` includes `target.h` .
3. `defines.h` includes `configSelect.h` .
4. `configSelect.h` includes either `config.h` or `configDebug.h` .
5. `defines.h` includes the selected pilot header or defines `PILOT_DEFAULT` .
6. `defines.h` includes `remote.h` , which includes exactly one selected remote header.
7. `defines.h` includes either `defines/defines_2-ad.h` for `REMOTE_AUTODETECT` , or dynamically includes `defines/defines_2-TARGET-LAYOUT.h` .
8. `defines.h` includes `setup.h` .
9. `defines.h` defines optional RTT logging macros.
10. `defines.h` validates the selected configuration and derives shared structs, defaults, and helper macros.

This chain lets the compiler know the target, the user configuration, the selected remote, the selected pilot, and the board pin map before compiling the actual implementation files.

5.1 `target.h`

`target.h` adapts different MCU libraries and peripheral APIs to common firmware names. It contains macros such as:

- `TARGET_nvic_irq_enable(...)`
- `TARGET_adc_software_trigger_enable(...)`
- `TARGET_dma_init(...)`
- target-specific DMA channel and IRQ handler aliases
- `pinMode` , `pinModePull` , `pinModeAF` , `digitalWrite` , `digitalRead`
- Arduino-style pin constants such as `PA15` , `PB6` , `PC14`

When adding or fixing a target, first look for the smallest useful `TARGET_*` abstraction. If a GD32F130 library call differs on GD32F103, STM32F103, GD32E230, or a future target, prefer adding a wrapper macro in `target.h` over duplicating code in `setup.c` or `it.c` .

5.2 `configSelect.h`

`configSelect.h` is intentionally a small indirection layer so the developer can keep private or experimental settings out of the normal public config. In this checkout it defines `CONFIG_DEBUG` , so `configDebug.h` is selected. For a normal public/default build path this layer can select `config.h` instead. Do not remove this layer.

5.3 `config.h`

`config.h` is the main end-user configuration file. It should remain readable for non-programmers. Important selections include:

- `REMOTE_AUTODETECT` , which changes the whole configuration path.
- `LAYOUT` and optional `LAYOUT_SUB` .
- `BLDC_BC` , `BLDC_SINE` , and optional `BLDC_SINE_BOOSTER` .
- `DRIVING_MODE` .
- exactly one of `MASTER` , `SLAVE` , or `SINGLE` .
- exactly one remote such as `REMOTE_DUMMY` , `REMOTE_UART` , `REMOTE_UARTBUS` , `REMOTE_CRSF` , `REMOTE_ROS2` , `REMOTE_ADC` , or `REMOTE_OPTIMIZEPID` .
- optional pilot code such as `PILOT_USER` or `PILOT_HOVERBIKE` .
- UART assignment through `MASTERSLAVE_USART` and `REMOTE_USART` .
- battery, current, timeout, buzzer, button, and charger-related settings.

The user should not need to edit random source files to use a new normal feature. Add a clear define to `config.h` when a feature is intended to be user-selectable.

5.4 Dynamic layout include

For normal firmware builds, `defines.h` builds the layout include path from `TARGET` and `LAYOUT` :

```
#define STRINGIZE_AUX(a) #a
#define STRINGIZE(a) STRINGIZE_AUX(a)
#define INCLUE_FILE(target,version) STRINGIZE(defines/defines_2-target-version.h)
#include INCLUE_FILE(TARGET , LAYOUT)
```

The macro name is currently spelled `INCLUE_FILE` in the code. Do not casually rename it unless you update all uses and verify all supported build paths.

5.5 `REMOTE_AUTODETECT`

`REMOTE_AUTODETECT` is a special remote and a special configuration mode. It includes `defines/defines_2-ad.h`, forces `SINGLE` / `MASTER_OR_SINGLE`, uses block commutation, and drives a simple UART or RTT user interface for board identification.

Autodetect is one of the largest and most complex remote modules and can approach the 32 kB flash limit. Treat it as a special tool for discovering unknown boards, not as a normal runtime feature.

6. Board layout files

Layout files live in `HoverBoardGigaDevice/Inc/defines/` and follow the pattern:

```
defines_2-TARGET-LAYOUT.h
```

For example:

- `defines_2-1-20.h` : target 1 / GD32F130, layout 20.
- `defines_2-2-1.h` : target 2 / GD32F103, layout 1.
- `defines_2-ad.h` : autodetect mode.
- `defines_2-x-y.h` : template/example for new layouts.

Layout files should define hardware facts:

- LED pins such as `LED_GREEN`, `LED_ORANGE`, `LED_RED`, `UPPER_LED`, `LOWER_LED`.
- hall pins `HALL_A`, `HALL_B`, `HALL_C`.
- BLDC MOSFET pins `BLDC_GH`, `BLDC_GL`, `BLDC_BH`, `BLDC_BL`, `BLDC_YH`, `BLDC_YL`.
- `TIMER_BLDC_PULLUP` and optional emergency shutdown pin.
- ADC inputs such as `VBATT`, `CURRENT_DC`, or phase current pins.
- `ADC_BATTERY_VOLT` or `MOTOR_AMP_CONV_DC_AMP` if the board needs custom conversion.
- `SELF_HOLD`, `BUTTON`, `BUTTON_PU`, `CHARGE_STATE`, `BUZZER`, `MOSFET_OUT`.

- available UART pins, for example `USART0_TX`, `USART0_RX`, `USART1_TX`, `USART1_RX`, and target-2-only `USART2_TX/RX` where applicable.
- optional IMU and I2C selections such as `MPU_6050old` and `I2C_PB6PB7`.
- optional user hardware pins such as `PHOTO_L` and `PHOTO_R`.

Some physical boards have small variants without deserving a new layout ID. Use `LAYOUT_SUB` for this. Example: `defines_2-1-7.h` uses `LAYOUT_SUB` and `MASTER_OR_SINGLE` to decide which board owns the buzzer.

When adding a layout:

1. Start from `defines_2-x-y.h` or the closest known layout.
2. Keep the file mostly declarative. Avoid behavior code in layout files.
3. Document uncertain pins with comments.
4. Set only pins and constants that are actually known.
5. Verify UART conflicts with I2C and other alternate functions.
6. Verify ADC channel availability for the selected target.
7. Add `LAYOUT_SUB` logic only when the board variants are truly the same layout family.

7. Object-oriented-like architecture in C

The project is intentionally written in C, but it uses an object-oriented-like style. There are "base classes" as headers with common virtual function declarations, and selected implementation files provide those functions.

This is not dynamic OOP. The firmware does not create objects at runtime. Exactly one implementation is selected at compile time with preprocessor defines. This is intentional because:

- many MCUs have only 32 kB flash,
- each remote corresponds to real hardware or one serial protocol,
- each BLDC method corresponds to one motor-control strategy,
- there is usually no realistic use case for switching from ESP32 UART to joystick or from sine to block commutation at runtime.

Preserve this pattern.

8. Remote modules

`Inc/remote.h` declares the remote interface:

```
void RemoteCallback(void);
void RemoteUpdate(void);
```

Every selected remote implementation must provide both functions, even if one is empty.

Common pattern:

- `RemoteCallback()` is called from DMA/USART receive interrupt context through `it.c`.
- `RemoteUpdate()` is called from the main loop, usually every second loop iteration, about every 10 ms with the current `DELAY_IN_MAIN_LOOP`.
- Remotes write global command values such as `speed`, `steer`, `wState`, `iDrivingMode`, or timeout flags.
- UART remotes should use `ResetTimeout()` when valid commands are received.
- Lost serial control should result in safe stop behavior, for example through `LOST_CONNECTION_STOP_MILLIS`, `bRemoteTimeout`, or setting speed/steer to zero.

Existing remote modules include:

- `remoteDummy.c`: internal test/zigzag style remote and RTT interaction.
- `remoteUart.c`: one board controlled by a serial host.
- `remoteUartBus.c`: ESP32/Arduino as master on a shared UART bus with multiple boards and `SLAVE_ID`.
- `remoteCrsf.c`: CRSF receiver input.
- `RemoteROS2.c`: ROS2-style serial command/feedback path.
- `RemoteAdc.c`: analog speed/steer input on PA2/PA3 and calibration through button long press.
- `RemoteAutodetect.c`: board pin discovery UI and scanning logic.
- `remoteOptimizePID.c`: PID optimizer/test remote.

When adding a new remote:

1. Add `Inc/remoteXY.h` and `Src/remoteXY.c`.
2. Guard the implementation with `#ifdef REMOTE_XY`.
3. Add the include branch to `remote.h`.
4. Add a commented selection block and any user settings to `config.h`.
5. Define `REMOTE_BAUD` in the remote header if it uses UART and needs a default.
6. Use `REMOTE_USART` assignment in `config.h`, not hard-coded USART names.
7. Keep byte parsing interrupt-safe. Do not do blocking work in `RemoteCallback()`.
8. Keep protocol structs packed and mirrored in the external examples if the protocol is used by ESP32/Arduino.
9. Use `CalcCRC()` or the protocol's existing checksum style consistently.

10. Add or update an example under `Arduino Examples/` or `PlatformIO Examples/` if external users need to drive it.

9. BLDC modules

`Inc/bldc.h` is the BLDC base interface. It declares core functions implemented in `bldc.c` and the virtual motor-control hooks:

```
void InitBldc(void);
void bldc_get_pwm(int pwm, int pos, int *y, int *b, int *g);
```

The selected implementation is included through:

- `BLDC_BC -> bldcBC.h / bldcBC.c`
- `BLDC_SINE -> bldcSINE.h / bldcSINE.c`

`bldc.c` owns the shared, high-frequency BLDC calculation:

- ADC current and battery voltage processing.
- hall sensor readout and hall-to-position mapping.
- current limiting and enable/timeout handling.
- input interpretation through `Driver()`.
- low-pass filtering of PWM command.
- calling `bldc_get_pwm()` to get phase PWM values.
- updating timer compare registers.
- odometer and speed estimation.
- optional buzzer generation.
- optional `PilotCalculate()` hook when a pilot requires a fast calculation path.

`CalculateBLDC()` is called from the ADC DMA interrupt path. It is time-critical. Avoid expensive additions here.

When adding a new BLDC method such as future `BldcFOC.c`:

1. Add a new `BLDC_FOC` or similar config define.
2. Add `Inc/bldcFOC.h` and `Src/bldcFOC.c`.
3. Implement `InitBldc()` and `bldc_get_pwm(...)`.
4. Extend the selection block in `bldc.h`.
5. Keep common ADC, hall, driver, filtering, safety, and timer register writes in `bldc.c` unless there is a strong reason to generalize the base interface.

6. Be extremely careful with flash size, ISR time, integer overflow, and target compatibility.

10. Driving modes and `driver.c`

`DRIVING_MODE` is selected in `config.h`, but runtime switching can make sense. For that reason the driving-mode interpretation is not implemented as separate OOL modules.

`SetBldcInput()` in `bldc.c` maps input values according to the active mode:

- `0` : raw PWM, roughly `-1000..1000`.
- `1` : speed in `revs/s * 1024`.
- `2` : torque in `Nm * 1024`.
- `3` : odometer/position in hall steps.

`driver.c` owns the integer PID controller and maps driving-mode input to a PWM command. PID defaults are defined in `defines.h` through `PIDINIT_a3o`, with comments explaining each parameter. If changing PID behavior, verify `REMOTE_OPTIMIZEPID`, speed mode, and position mode assumptions.

11. Pilot modules

Pilots are optional onboard application logic. They are the right place for user features that are neither a low-level remote input nor a motor-control algorithm.

Examples:

- `PilotUser.c` : template for custom user code.
- `PilotHoverbike.c` : experimental pedal-assist / hoverbike logic.

The normal hook is:

```
void Pilot(int16_t* pPwmMaster, int16_t* pPwmSlave);
```

If no pilot is selected, `defines.h` defines `PILOT_DEFAULT`, and `main.c` uses the default speed/steer mixing with `SPEED_COEFFICIENT` and `STEER_COEFFICIENT`.

A pilot can also define `PILOT_CALCULATE` and implement `PilotCalculate()`, which is then called from `CalculateBLDC()`. Use this only when the pilot truly needs the high-frequency motor-control timing. Most pilot code should stay in `Pilot()`, called from the main loop.

When adding a pilot:

1. Add `Inc/PilotXY.h` and `Src/PilotXY.c` .
2. Guard it with `#ifdef PILOT_XY` .
3. Add the include branch to `defines.h` .
4. Add a commented selection line to `config.h` .
5. Define needed hardware pins in layout files, not inside the pilot.
6. Initialize those pins in `setup.c` only if no existing generic pattern covers them.
7. Use `bPilotTimeout` to request safe motor disable when the pilot logic loses confidence.

12. Master, slave, and single firmware roles

Gen1 boards often had two motors on one control board. Gen2 split hoverboards have two boards, each driving one motor, connected by a master/slave UART link in the original hoverboard.

The firmware must know which role it is compiled for:

- `MASTER` : receives or computes global commands, drives its own motor, and sends slave commands.
- `SLAVE` : receives commands from the master and drives one motor.
- `SINGLE` : standalone board without the original master/slave pair.

`MASTER_OR_SINGLE` is used for logic that belongs to a controller that accepts remote input and owns user-facing behavior. `MASTER_OR_SLAVE` is used for logic that belongs to the original master/slave UART pair.

`RemoteUartBus` allows several `SINGLE` boards on one UART bus, controlled by an external ESP32/Arduino master. In that mode each board needs a unique `SLAVE_ID` .

13. UART and serial protocols

UART ownership is configured in `config.h` :

```
#define MASTERSLAVE_USART 1
#define REMOTE_USART      0
```

`defines.h` validates that master/slave and remote USARTs do not collide and then maps them to `HAS_USART0` , `HAS_USART1` , `HAS_USART2` , `USART_REMOTE` , `USART_MASTERSLAVE` , and the matching one-byte DMA receive buffers.

`setup.c` configures USARTs and DMA receive channels. `it.c` routes DMA receive completion to either:

- `RemoteCallback()` for the selected remote, or
- `UpdateUSARTMasterSlaveInput()` for master/slave communication.

The external examples mirror the firmware protocol structs. These structs often use packed layout on the host side. Be careful when changing field order, field size, start bytes, CRC rules, or endian assumptions.

Important protocol notes:

- `REMOTE_UART` and `REMOTE_UARTBUS` default to 19200 baud unless overridden, partly because simple Arduino SoftwareSerial cannot reliably do 115200.
- Some ESP32 examples use 115200 baud; the firmware remote header/config must match.
- `REMOTE_UARTBUS` uses a shared serial bus and a target `SLAVE_ID`.
- `REMOTE_UARTBUS` has special open-drain/no-pullup behavior for TX in `setup.c` when selected; external controllers may need to provide pullups.
- Feedback frames commonly start with `START_FRAME 0xABCD`.
- Command frames in the legacy UART path often use `'/'` as a start byte.
- `CalcCRC()` in `comms.c` is shared by several protocols.

14. Timing and interrupts

`it.c` centralizes interrupts.

Main timing paths:

- `SysTick_Handler()` increments `msTicks` every millisecond.
- `TIMEOUT_IrqHandler()` runs the command timeout timer where supported.
- `TARGET_TIMER0_BRK_UP_TRG_COM_IRQHandler()` handles the BLDC timer update interrupt.
- The BLDC timer update ISR triggers the ADC conversion.
- `TARGET_DMA_Channel0_IRQHandler()` runs when the ADC DMA scan is complete and calls `CalculateBLDC()`.
- USART DMA handlers call `RemoteCallback()` or `UpdateUSARTMasterSlaveInput()`.

The BLDC timer uses center-aligned PWM. Some libraries/hardware trigger update interrupts on both up and down counting even when configured differently. The current code uses a static toggle in the timer ISR and only triggers ADC every second call. This was a target-cross-compatible solution for GD32 and STM32 differences. Do not simplify it unless you verify all supported targets on hardware.

Interrupt priority intent:

- BLDC timer / hall-related work must stay very high priority.
- ADC DMA / `CalculateBLDC()` must be fast and deterministic.
- USART receive parsing must not interrupt the motor-critical path.
- Timeout and general main-loop work are lower priority.

Do not add blocking calls, long loops, serial printing, flash writes, or complex parsing in these ISRs.

15. `setup.c` responsibilities

`setup.c` contains low-level hardware setup:

- watchdog setup,
- timeout timer setup,
- GPIO mode setup,
- PWM timer setup,
- ADC and DMA setup,
- USART and DMA setup,
- flash-based config read/write,
- clock initialization.

Prefer extending existing helper patterns in `setup.c` rather than adding target-specific code everywhere. For target differences, first add `TARGET_*` wrappers in `target.h`.

The flash config storage uses `ConfigData` from `defines.h`, with versioning through `EEPROM_VERSION`. If changing `ConfigData`, update the version and keep word alignment/padding in mind.

16. Safety behavior

Safety-related behavior exists in several layers:

- `DC_CUR_LIMIT` stops PWM when DC current is too high.
- invalid hall state `0b000` or `0b111` disables PWM in `CalculateBLDC()`.
- `timedOut`, `bRemoteTimeout`, and `bPilotTimeout` can disable output.
- `CHARGE_STATE` can prevent driving while charging unless disabled for testing.
- `BATTERY_LOW_SHUTOFF` can shut down under low voltage.
- `SELF_HOLD` controls board power latch where available.
- `BUTTON` can request shutdown.

- watchdog reloads keep the firmware honest.

Do not remove or bypass these checks for convenience. If a test mode needs relaxed behavior, put it behind an explicit config define and comment the danger.

`REMOTE_AUTODETECT` can drive a motor without normal hall input while finding pins. Test it only with a current-limited supply, typically 1-2 A. A charger with about 1.5 A may be usable for low-risk detection, but a real battery can make mistakes destructive.

17. Memory and performance constraints

The firmware often runs on small Cortex-M MCUs, sometimes with only 32 kB flash and no floating-point unit.

Guidelines:

- Avoid dynamic memory allocation.
- Avoid pulling in large standard library features.
- Avoid `printf` in production paths; float printf is especially expensive.
- Avoid big lookup tables unless they replace more expensive runtime work and are needed only in selected builds.
- Keep alternate implementations behind compile-time defines.
- Keep floats out of high-frequency interrupt code where reasonably possible.
- Prefer fixed-point or integer math for control loops.
- Check map/size output after large features.
- Keep debug helpers behind `#ifdef DEBUG_*`, `RTT_REMOTE`, or a feature-specific define.

18. External ESP32/Arduino examples

When changing UART protocols, inspect both firmware and examples:

- `Arduino Examples/TestSpeed/hoverserial.h`
- `Arduino Examples/Esp32_PPM/hoverserial.h`
- `Arduino Examples/Test_4Wheeler/hoverserial.h`
- `Arduino Examples/TestROS2/ros2serial.h`
- `PlatformIO Examples/TestSpeed/include/hoverserial.h`
- `PlatformIO Examples/HoverBike CYD LVGL/include/hoverserial.h`

The examples are not just demos; they document expected host-side packet structures. Keep them compatible or update them deliberately with the firmware protocol change.

19. Preferred locations for common feature requests

Use this map before editing:

- New serial controller protocol: add `RemoteXY.c/.h` .
- New RC receiver protocol: add `RemoteXY.c/.h` .
- New test input source: add a remote, often based on `remoteDummy.c` .
- User application behavior such as e-bike pedal assist: add `PilotXY.c/.h` .
- New motor-control commutation strategy: add `bldcXY.c/.h` .
- New board pinout: add `defines/defines_2-TARGET-LAYOUT.h` .
- New board variant with only small differences: add `LAYOUT_SUB` handling.
- New MCU family: extend `target.h` , build files, startup/linker/vendor files, and add target-specific layout files.
- UART packet format used by ESP32: update the matching example `hoverserial.h` too.
- Runtime driving mode behavior: look at `driver.c` , `SetBldcInput()` , and `DRIVING_MODE` .
- Flash-stored calibration/config: update `ConfigData` , `EEPROM_VERSION` , and `ConfigReset()` .

20. Pull request expectations

Good changes should:

- fit the existing OOL structure,
- be selected by clear config defines,
- avoid unrelated refactors,
- compile for the affected target(s),
- keep existing remotes and layouts working,
- document hardware assumptions,
- preserve safety behavior,
- keep flash size realistic,
- include or update an external example when protocol users need it.

Changes to `main.c` , `bldc.c` , or `it.c` should be reviewed especially carefully. They are acceptable when they fix real bugs, improve performance, improve the extension architecture, or are required by a feature that cannot live anywhere else.

21. Verification checklist for AI agents

Before claiming a firmware code change is done:

1. Identify the active target, layout, role, remote, pilot, and BLDC method.
2. Check that only one remote and one BLDC method are selected.
3. Compile at least the affected PlatformIO environment if PlatformIO dependencies are available:
   - `pio run -e genericGD32F130C8`
   - `pio run -e genericGD32F103C8`
   - `pio run -e genericGD32E230C8`
4. If PlatformIO is unavailable, at least compile syntax where possible and state what could not be verified.
5. For Keil-only or target-startup changes, mention that Keil/hardware validation is still needed.
6. For UART protocol changes, build or at least inspect the matching external example.
7. For motor-control changes, review interrupt runtime, current limit behavior, hall invalid behavior, and enable/timeout behavior.
8. For layout changes, verify pin alternate functions and ADC channel availability.
9. For config storage changes, bump `EEPROM_VERSION` and verify struct alignment.
10. For safety-sensitive changes, describe the hardware test conditions.

22. Future wishes that fit the architecture

Known future directions include:

- `BldcFOC.c` or another FOC-style BLDC implementation.
- A clean `RemoteBluetooth.c` based on the old removed Bluetooth support, but integrated as a normal remote.
- Better support for target 4 and future targets such as MM32SPIN0x, LKS32, MM32SPIN25, and GD32E235.
- Stronger online compiler integration.
- More robust board autodetect results and layout generation.
- Better ESP32 tooling for multiple `SINGLE` boards on `REMOTE_UARTBUS`.

Do not start these just because they are listed here. They are architectural wishes, not permission to make broad unrelated changes.

23. Coding style notes

This codebase has a pragmatic embedded C style. Preserve it.

- Use C, not C++, unless the project direction explicitly changes.
- Keep function names and file naming close to existing patterns.
- Use preprocessor guards for selected modules.
- Prefer clear hardware comments over abstract comments.
- Keep ASCII text unless a file already needs non-ASCII.
- Avoid new dependencies.
- Avoid global renames.
- Avoid changing public packet structs casually.
- Keep includes minimal and consistent with the existing include chain.
- Keep debug variables if they are useful for STMStudio, McuViewer, RTT, or hardware diagnosis.

The firmware is compact, hardware-close, and built around compile-time selection. The best additions feel like they were always meant to be one more selected module in the existing system.